

# Dimension Shift: Sci-Fi Arena

## A procedural third-person wave shooter in Three.js

### Technical report and user manual

Mattia Cosimi — Matricola 2278125

Interactive Graphics — Prof. Marco Schaerf — Sapienza University of Rome

---

## Contents

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                            | <b>2</b>  |
| <b>2</b> | <b>Development environment</b>                 | <b>2</b>  |
| <b>3</b> | <b>Libraries, tools and external resources</b> | <b>3</b>  |
| <b>4</b> | <b>Technical aspects</b>                       | <b>3</b>  |
| 4.1      | Hierarchical models . . . . .                  | 3         |
| 4.2      | Lighting and shadows . . . . .                 | 4         |
| 4.3      | The sky . . . . .                              | 5         |
| 4.4      | Textures of different kinds . . . . .          | 5         |
| 4.5      | The Dimension Shift . . . . .                  | 5         |
| 4.6      | Camera . . . . .                               | 6         |
| 4.7      | Projectiles and predictive aiming . . . . .    | 6         |
| 4.8      | Collision system . . . . .                     | 7         |
| 4.9      | Hand written animations . . . . .              | 7         |
| 4.10     | Enemy AI and the wave system . . . . .         | 8         |
| 4.11     | Post processing pipeline . . . . .             | 8         |
| 4.12     | Render loop . . . . .                          | 9         |
| <b>5</b> | <b>Implemented interactions</b>                | <b>9</b>  |
| <b>6</b> | <b>User manual</b>                             | <b>9</b>  |
| <b>7</b> | <b>Architecture and design choices</b>         | <b>10</b> |
| 7.1      | Modules and the base class pattern . . . . .   | 10        |
| 7.2      | Context objects . . . . .                      | 10        |
| 7.3      | Frame rate independence . . . . .              | 10        |
| 7.4      | Two lighting models in one scene . . . . .     | 11        |
| 7.5      | Robustness . . . . .                           | 11        |

---

# 1 Introduction

This document describes *Dimension Shift: Sci-Fi Arena*. The game is a third-person arena shooter: the player chooses one of two combat classes and has to survive ten escalating waves of enemies inside a walled sci-fi arena, the last of which is a fight against a boss, DOOMHORN.

The project was built entirely from scratch. Every model, every animation and every gameplay behaviour is constructed through code, without importing animated models and without using an external physics engine.

The distinctive element of the game is the **Dimension Shift**. Pressing a single key re renders the whole scene in a completely different visual style, switching between a realistic physically based look and a flat, pixelated cartoon look. The shift is not a cosmetic filter: it changes how much damage enemies take and it reveals power ups that are otherwise invisible, so it is a resource the player has to manage during a fight.

# 2 Development environment

The project is developed with **Three.js** (version 0.160), a high level JavaScript library built on top of **WebGL**. WebGL is the browser graphics API that performs GPU accelerated 3D rendering through shaders executed on the graphics card; Three.js hides its low level details behind high level abstractions for scenes, cameras, lights, materials and geometries, so that almost no raw WebGL code has to be written; the one shader written directly in GLSL is the short one that draws the sky (Section 4.3).

The game has no back end: no application server, no database, no server side logic. Rendering, the physics, the wave rules and the input handling are all JavaScript executed locally in the user's browser. The code is organised as a set of native ES modules, each with a single responsibility, wired together by an `importmap` declared in `index.html` and resolved directly by the browser. There is consequently no build step, no Webpack, no Vite, no bundling, and Three.js itself is copied into the repository (`libs/three/`) rather than loaded from a CDN, so the project has no external dependency and runs offline.

The one deployment requirement is a web server, needed only to *serve* the files: browsers refuse to load ES modules over the `file://` protocol, so `index.html` cannot be opened directly from disk and the folder must be delivered over HTTP. Any static file server is enough, and the public build is hosted on GitHub Pages.

| File                                | Responsibility                                                                                                                         |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <code>index.html / style.css</code> | Interface structure and style: menu, HUD, pause / game over / victory overlays                                                         |
| <code>main.js</code>                | Renderer, camera, render loop, input, wave manager, collision functions, bullet simulation, HUD wiring                                 |
| <code>environment.js</code>         | Arena construction: ground, perimeter walls, corner beacons, crates, jump platforms, skybox, procedural textures, height query helpers |
| <code>dimensionShift.js</code>      | The realistic ↔ toon material and rendering swap, and its timers                                                                       |
| <code>weapons/</code>               | Weapon base class → Rifle, Pistol, SniperRifle                                                                                         |
| <code>playerClasses/</code>         | PlayerClass base class → Assault, Sniper                                                                                               |
| <code>enemies/</code>               | Enemy base class → Grunt, Shooter, Brute, Marksman, Boss                                                                               |
| <code>powerups/</code>              | PowerUp base class → health, armour (small / large), strength, dimension cache                                                         |

### 3 Libraries, tools and external resources

In accordance with the assignment, every component used but not developed is listed below.

| Resource                          | Type                             | Use                                                                                                                                                |
|-----------------------------------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Three.js (v0.160)                 | Library                          | WebGL-based 3D graphics engine (core)                                                                                                              |
| RoundedBoxGeometry                | Three.js add on                  | Bevelled boxes for the body parts of the player and the enemies                                                                                    |
| EffectComposer and passes         | Three.js add ons                 | Post processing pipeline:<br>RenderPass,<br>UnrealBloomPass,<br>RenderPixelatedPass,<br>OutputPass,<br>ShaderPass with<br>BrightnessContrastShader |
| metal_weave_011                   | Texture set (3dtextures.me, CC0) | Base colour, normal and roughness maps for the player's armour (tinted per class)                                                                  |
| frosted_glass (Glass Frosted 003) | Texture set (3dtextures.me, CC0) | Base colour, normal and roughness maps for the player's backpack (untinted)                                                                        |

**Important.** No 3D model and no animation were imported. All models (the player, the five enemy types, the weapons, the power ups and the whole arena) are built procedurally by combining Three.js geometric primitives (rounded boxes, cylinders, cones, spheres, planes). All animations are implemented in JavaScript, computed directly from the elapsed time each frame; no interpolation or animation library is used. Apart from the two armour texture sets, every other texture (the arena floor grid, the toon shading ramp) is generated via code on an HTML canvas. No physics engine is used.

### 4 Technical aspects

#### 4.1 Hierarchical models

Every character in the game (the player and each of the five enemy types) is built as a tree of nested `THREE.Group` nodes, where each node is a body part.

The principle of the hierarchy is that a transformation applied to a parent node propagates automatically to all of its children. Each body is built so that its feet rest exactly on the ground: a root group holds the legs and the torso, and from the torso descend the head and the two arms. When an animation rotates a joint, the whole sub tree below it follows as a unit, without animating each segment separately.

The player model exploits this most directly by splitting the body into two independently oriented chains. The legs face the direction the player is walking, while the torso, a child of the player group, is rotated back out of the parent's orientation and into the camera's, so that the upper

body always tracks where the player is aiming. The torso also leans up and down with the vertical look angle. Because the equipped weapon is parented to the arm, which is parented to the torso, the gun inherits this twist and lean for free, with no dedicated code.

The player is the project's primary complex hierarchical model, both for the depth of its tree and because its animation is what most directly exploits that structure. The boss, DOOMHORN, is the largest model in the game. Its hierarchy nests a broad torso (carrying a glowing chest core and five spine spikes), a head (two emissive eyes and two horns), two wide set arms (each ending in a claw and an invisible marker node for the weapon spawn point), and two legs. The whole group is then scaled by a factor of four as a single operation: because the model's local origin already sits at ground level, it stays grounded, and every child transform, including the two weapon markers read during the attack, scales with it.

## 4.2 Lighting and shadows

The scene contains eight light objects at all times. They combine three different types, because a single source is never enough: the renderer does not compute indirect light, so anything a lamp does not hit directly would simply be black. The set is built around one clear main light, with the others filling in what it leaves dark.

- The **hemisphere light** is the base ambient. It blends a cool colour from the sky above with a warmer one bounced off the ground below, so up facing and down facing surfaces are tinted slightly differently and the scene never falls to pure black in the corners the other lights miss. It is kept weak in the realistic dimension, where it only has to lift the darkest areas, and it costs almost nothing since a hemisphere light casts no shadows; in the toon dimension it is turned up and saturated, and becomes the main coloured light of that daytime look.
- The **directional light** is the key light: parallel rays from one fixed direction, tinted a cold blue to match the night arena mood. Being directional, it lights the whole arena consistently wherever the player stands, it is what gives objects a lit side and a shadowed side, and it is the only light that casts shadows. Those ground shadows are what anchor the characters to the floor and make jump height and position readable.
- Two **point lights** act as fills for what the key light leaves in shadow. One is fixed on the opposite side of the arena, cool toned, so the dark side of every object is lifted a little instead of going flat. The other is moved onto the camera every frame: since the directional light always lights the same world space side, the side the player is looking at can turn away from it as they move around, and a fill attached to the camera keeps whatever is on screen lit regardless of where they stand or face. It is deliberately short range, so it only rescues nearby detail rather than washing the whole scene out.
- The last four **point lights** sit one inside each corner beacon. These are atmospheric: they make the glowing beacon props read as actually emitting light, they mark the corners of the arena as landmarks, and the four pools of red colour keep the large open space from looking flat and evenly lit.

The shadows use a technique called shadow mapping. To make them appear, shadows must be switched on in three places at the same time: once on the renderer, once on the light that is allowed to cast them, and once on each mesh. A mesh in fact has two separate settings: one to say that it casts a shadow on other objects, and one to say that shadows from other objects are drawn on its own surface. The renderer is set to the `PCFSoftShadowMap` mode, which gives soft, slightly blurred shadow edges instead of hard, jagged ones.

A directional light draws its shadows inside a box, and by default that box is only a few units wide. The arena is much larger than that, so most of it would receive no shadow at all. The box is therefore made wide enough to cover the whole playable area. Enlarging it has one side effect: the shadow becomes less precise and surfaces can start to shadow themselves by mistake, so a

small correction value (a depth bias) is added to remove that.

Only the directional light casts shadows. Every shadow casting light needs its own extra render of the scene each frame, so using all eight lights for shadows would be far more expensive than the small visual gain at this scale.

### 4.3 The sky

The background is not a flat colour or a downloaded panorama but a large inverted sphere drawn by a short custom shader written directly in GLSL. The fragment shader takes the interpolated world position of each point on the sphere, reads its height, and blends between a horizon colour and a zenith colour accordingly, producing a smooth vertical gradient. The two colours are passed to the shader as uniforms, which is what lets the Dimension Shift (Section 4.5) repaint the entire sky in one step by writing new values into them. The horizon colour is matched to the fog colour, so the ground dissolves seamlessly into the sky at the far distance. A field of eight hundred points scattered on a surrounding shell adds stars in the realistic dimension; they are hidden in toon, where the sky reads as daytime.

### 4.4 Textures of different kinds

The assignment explicitly requires textures of different kinds, citing colour, normal and specular as examples. The project uses all three, on the player's armour, through a `MeshStandardMaterial` (the physically based, PBR material).

- The **colour map** (base colour) defines the surface colour pixel by pixel. It is loaded as an sRGB texture and then tinted per body part through the material's colour, so that a single greyscale metal texture yields a differently coloured torso, limbs, head and trim, and a different tint per class.
- The **normal map** encodes, in each pixel, a perturbed surface normal. Under the lights this simulates the surface relief of the woven metal without adding a single polygon: the geometry stays a plain bevelled box and all the surface detail is an optical effect on the lighting. It is kept in linear colour space, because it is data and not colour.
- The **roughness map** is the modern PBR equivalent of the classic specular map. It controls, per pixel, how glossy each part of the surface is, so that raised strands catch a sharper highlight than the recesses.

The UV tiling is tuned per body part so that the weave reads at a consistent physical size across the wide torso and the narrow limbs. The backpack uses a second, deliberately different texture set: a frosted glass, left untinted so its own appearance shows through.

The rest of the scene's textures are generated procedurally on an HTML canvas: the arena floor is a repeating grid drawn in code, and the toon shading ramp (see below) is a one dimensional gradient. Finally, every sci-fi accent in the game (visors, chest cores, weapon glow, power up shells, the corner beacons, the boss's eyes) is an *emissive* material: a flat bright colour that is not lit by any of the eight real lights but is turned into a convincing glow by the bloom post process.

### 4.5 The Dimension Shift

The Dimension Shift is the game's signature technical feature. A single key toggles the whole scene between two coherent presentation styles.

|                  | Realistic (default)                                       | Toon                                                                                                            |
|------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Materials        | physically based<br>( <code>MeshStandardMaterial</code> ) | cel shaded ( <code>MeshToonMaterial</code> ),<br>lighting quantised into a few flat<br>bands by a gradient ramp |
| Tone mapping     | ACES Filmic                                               | none                                                                                                            |
| Post processing  | bloom                                                     | pixelation with an outline pass;<br>bloom off                                                                   |
| Lights, sky, fog | cool night preset                                         | warm, bright “daytime comic” preset                                                                             |

The switch is implemented by snapshotting the realistic preset the first time it runs and then, on every toggle, rewriting the hemisphere and directional lights’ colour and intensity, repainting the sky shader gradient and the fog, toggling the tone mapping and the relevant post processing passes, and replacing every mesh’s material with its counterpart in the other style. A per frame synchronisation pass re applies the current style to the whole scene, so that anything spawned mid shift (a new bullet, a new enemy, a new power up) appears in the correct style automatically instead of in the wrong one.

The shift carries real gameplay stakes rather than being purely visual. Enemies take fifty percent more damage while the player is in the toon dimension, and a special class of power up (which restores both health and armour) is fully transparent in the realistic dimension and only becomes visible and collectible in toon. The shift is limited: fifteen seconds active, then an automatic revert, then a longer cooldown before it can be triggered again, with a denial flash if the player presses the key too early. It is therefore a window to be spent deliberately, not a mode to leave on.

## 4.6 Camera

The gameplay camera is a third person orbit. Its position is expressed in spherical coordinates around a pivot just above the player: a yaw and a pitch angle, both driven by the mouse with the pointer locked, and a fixed distance. Every frame the camera is placed behind the pivot along its own view direction, pulled back by that distance and then shifted slightly sideways and down in screen space, which gives the over the shoulder framing; the shift is computed in the camera’s own right/up axes rather than a flat horizontal guess, so it stays a consistent screen space offset no matter how far the view is tilted. The pitch is clamped to a range (about  $-17^\circ$  to  $+49^\circ$ ) that still lets the player look up at an enemy on a platform or down at one below, and a lower bound on the camera’s height keeps it from dipping below the ground at extreme angles. The camera then looks along the exact same direction vector used to position it, so the screen centre is always a meaningful aim point; the clamps move the camera without ever desynchronising it from where shots are fired.

On the main menu a separate fixed camera frames the class preview model while it turns on a slow turntable.

## 4.7 Projectiles and predictive aiming

Every shot, from the player or from an enemy, is a small emissive sphere with a short trailing streak, tracked in a plain list and advanced each frame by its velocity vector. Player shots and enemy shots are kept in two separate lists, so enemies can never damage one another with a stray bullet, and each bullet also carries an age and is removed once it outlives its weapon’s range, hits the ground, hits an obstacle or crosses the arena wall.

A refined element here is the aiming on both sides. Ranged enemies do not fire at the player’s current position; they solve for where the player *will be* when the bullet arrives, by estimating

the travel time (distance divided by bullet speed) and offsetting the aim point by the player's measured velocity times that time. The player's crosshair does the mirror image: it measures each enemy's velocity from its frame to frame displacement and leads a strafing target the same way, so that a shot lined up on a moving enemy actually connects instead of passing behind it. A light auto aim assist nudges the shot toward whatever the crosshair is resting on.

## 4.8 Collision system

All collision is handled as two dimensional circle mathematics on the ground plane, plus a height rule for the surfaces that can be stood on.

Each crate, corner beacon and jump platform contributes one collision circle. A candidate position is rejected if the distance between its centre and any obstacle's centre is smaller than the sum of the two radii. Jump platforms are ignored once the entity is standing on top of them, which is what lets the player and the enemies walk freely across a platform instead of being blocked by its base.

Movement is resolved one axis at a time: the horizontal and the depth components of a step are tested separately, each with the other held fixed. This lets a character that walks into an obstacle slide along its edge in the direction that is still free, instead of stopping dead. As a guaranteed fallback for the cases where sliding cannot help (walking straight into an obstacle with no sideways component), a depenetration pass runs every frame and pushes any still overlapping entity straight out along the line from the obstacle's centre by exactly the overlap amount. The arena's perimeter wall has no real collider; instead, the player's and every enemy's position is clamped to a fixed boundary on both axes.

Bullets are treated as points for these tests. Enemy versus bullet and player versus bullet hits are simple distance checks against a per entity radius. Each enemy in fact carries two radii: one for how large a target it is to incoming fire, and a separate, usually equal one for how much clearance it needs from obstacles while moving. For the boss the two differ substantially: it is a large target but routes around obstacles on the small footprint of its planted feet, so that its oversized body is allowed to visually overlap a crate without the boss believing it is blocked.

## 4.9 Hand written animations

All animations are implemented in JavaScript and computed from the real time elapsed since the previous frame; nothing is imported or keyframed, and no tweening library is used.

The walk cycle, shared by the player and every enemy, drives the limbs from a single sine wave whose phase only advances while the character is actually moving. Opposite limbs are given opposite phase, and each arm is put in counter phase with the leg on the same side, reproducing a natural gait. The rotation is applied to the hip and shoulder joints of the hierarchy, so the whole limb swings as one.

The player's upper body is animated on top of this: as described in Section 4.1, the torso twists toward the aim direction and leans with the look pitch, over the leg motion, so that the character can run in one direction while aiming and shooting in another.

Jumping and falling use one shared vertical model for the player and for every enemy: a vertical velocity that gravity pulls down every frame, and a ground test that snaps the character back onto whatever surface is below it (the arena floor, or the top of a crate or platform) and marks it grounded. The jump strength and gravity are chosen so the apex is exactly two units, and the enemies reuse the identical numbers and the identical platform system, so an enemy can climb onto exactly the surfaces the player can, and starts falling on its own the moment it walks off one.

Other objects are animated too: the power ups spin and bob on a sine wave while floating, the strength buff surrounds the player with two counter rotating rings, and the camera follows the player on a smoothed orbit, while on the main menu the class preview model turns slowly on a turntable.

#### 4.10 Enemy AI and the wave system

Each enemy runs the same per frame update, and branches on whether it is still farther from the player than its attack range.

While approaching, it heads toward the player but bends its path by a small fixed angle, unique to that enemy, which fades to zero as it nears attack range. The effect is that a group of enemies fans out into a loose surround instead of all converging on the same point and queueing up single file, while the final approach still converges cleanly. If an obstacle lies directly ahead, the enemy either jumps onto it, when it is a climbable crate or platform within jump height, or steers hard around it, consistently to its own fixed side so the manoeuvre is smooth.

Once in range, the enemy does not stand still: most types strafe sideways at reduced speed, which reads as more alive and is harder to shoot, while one type is a fragile long range sniper that kites: it backs straight away if the player closes inside a set distance. The enemy always turns to face the player, animates its walk, and fires its attack on a cooldown. The attack itself is the one behaviour each subclass provides: a melee type deals damage directly on contact, a ranged type spawns a predicted aim bullet, and the boss fires one bolt from each hand with a genuine three dimensional aim vector, since its scaled up hands sit well above the player and a level shot would pass overhead.

There are five enemy types, summarised below.

| Type     | HP   | Speed | Damage | Range | Behaviour                                               |
|----------|------|-------|--------|-------|---------------------------------------------------------|
| Grunt    | 30   | 5.5   | 8      | 1.3   | Fast, fragile; rushes and hits on contact               |
| Shooter  | 45   | 3.0   | 6      | 13    | Stops and fires single bolts; holds position            |
| Brute    | 90   | 3.5   | 18     | 1.6   | Heavy and slow; hits hard in melee                      |
| Marksman | 35   | 3.2   | 16     | 18    | Glass cannon; kites, backing away inside 8 units        |
| Doomhorn | 1500 | 4.2   | 26     | 15    | Boss; $\times 4$ scale; one bolt from each hand, 3D aim |

The game has ten waves. The nine regular waves grow in size (from 8 up to 31 enemies, trickle spawned), and the composition shifts with the wave number: the Shooter's share of each spawn rises from 30% toward a 55% cap while the Grunt's falls, and the Brute and the Marksman are layered in gradually on top, each ramping from 5% to a 25% cap. The tenth wave is the boss alone.

#### 4.11 Post processing pipeline

Rather than drawing straight to the screen, the renderer feeds an `EffectComposer` chain. A render pass draws the scene; a pixelation pass (with a normal and depth based outline) is enabled only in the toon dimension; a bloom pass, which scans the finished image for bright pixels and blurs a halo out from them, is enabled only in the realistic dimension and is what makes all the emissive accents glow; an output pass converts the linear result back to display colour space; and a final shader pass applies a small brightness and contrast lift.



## 4.12 Render loop

The heart of the program is a render loop called every frame through `requestAnimationFrame`. Each iteration computes the delta time, updates the player and camera, advances the bullets, runs the enemy AI and the enemy bullets, updates the power ups, re synchronises the scene to the current dimension, and finally renders through the post processing chain. A game state (menu, in game, paused, victory) decides which of these updates run: on the menu, for instance, only the turntable preview updates and gameplay input is ignored. A resize handler keeps the camera's aspect ratio and the renderer's size matched to the browser window, so the scene is never stretched or cropped. Because every update is scaled by the delta time and not by the frame count, the game runs at the same real speed on fast and slow machines.

## 5 Implemented interactions

The game offers several forms of user interaction, covering the assignment's requirement broadly.

- **Class and weapon selection:** through the initial menu the player picks one of two combat classes, each with its own two weapon loadout and colour scheme, previewed on a rotating model.
- **Movement and aim:** full ground movement with the keyboard, free mouse look with pointer lock, and a jump. The vertical aim angle is clamped to a range that still allows shooting up at an enemy on a platform or down at one below.
- **Shooting:** the left mouse button fires; automatic weapons fire while held, the semi automatic pistol once per click. The two weapons are swapped with the number keys.
- **Dimension Shift:** one key toggles the rendering style, with the gameplay consequences described in Section 4.5.
- **Pause:** a key pauses the game and releases the mouse pointer; the frozen scene keeps rendering.
- **Camera inspection:** the mouse orbit lets the player look at the arena and their character from any horizontal angle while playing.

The interface reacts continuously: health and armour bars, the current wave and weapon, a dedicated boss health bar during the final wave, the Dimension Shift state as an active timer or a cooldown, wave announcement banners, and full screen overlays for pause, game over and victory.

## 6 User manual

**Goal.** Survive all ten waves and defeat the boss. Dying at any point ends the run.

**Starting.** From the main menu, click a class card and then PLAY. Clicking the game view locks the mouse pointer; the pause key releases it.

| Input                | Action                                  |
|----------------------|-----------------------------------------|
| W A S D / arrow keys | Move                                    |
| Mouse                | Look and aim                            |
| Space                | Jump                                    |
| Left mouse button    | Shoot (hold for automatic weapons)      |
| 1 / 2                | Switch weapon                           |
| TAB                  | Dimension Shift (realistic ↔ toon)      |
| Esc                  | Pause                                   |
| N                    | Skip the current wave (development aid) |

**Classes and weapons.** The two classes have identical movement (the same speed, 100 health,

and armour capped at 100 and starting empty); they differ only in their loadout and colour.

| Weapon       | Interval | Bullet speed | Damage | Mode           | Class   |
|--------------|----------|--------------|--------|----------------|---------|
| Rifle        | 0.20 s   | 60           | 10     | automatic      | Assault |
| Pistol       | 0.35 s   | 75           | 7      | semi-automatic | both    |
| Sniper Rifle | 1.10 s   | 130          | 45     | automatic      | Sniper  |

**Power ups.** These appear on the field during a run.

| Pick up                         | Effect                                                                        |
|---------------------------------|-------------------------------------------------------------------------------|
| Health (green cross)            | Restores health to full                                                       |
| Armor +25 (blue shield, small)  | Adds 25 armour, up to the cap                                                 |
| Armor full (blue shield, large) | Restores armour to full                                                       |
| Strength (orange)               | 15 s buff: $\times 1.6$ damage and $\times 1.3$ move speed                    |
| Dimension cache (violet)        | Full health <i>and</i> full armour (only visible while in the toon dimension) |

Armour absorbs incoming damage before health.

**How to play.** Move to keep distance from the melee enemies and break the line of fire of the ranged ones, using the crates and platforms as cover and high ground. Shoot enemies to destroy them; the aim rests at the screen centre with a light assist. Trigger the Dimension Shift before a hard fight or a wave of tough enemies: while in toon they take extra damage, and any hidden dimension cache power ups become collectible; but the shift is timed and then goes on cooldown, so spend it deliberately.

## 7 Architecture and design choices

Apart from the specific requirements, the project rests on a few architectural choices that are worth describing in their own right, because they are what keep the codebase tidy and easy to extend.

### 7.1 Modules and the base class pattern

The code is not one monolithic file but a set of modules with one task each, and within them the four families of game object (weapons, player classes, enemies and power ups) all follow the same pattern: a base class holds everything the family has in common (statistics, shared movement, the walk animation, taking damage), and each concrete subclass supplies only what differs, typically just its 3D model and one behaviour hook. The game loop never checks which concrete type it is holding: it calls the same method on every enemy and lets the right subclass respond. Adding a new enemy, weapon or power up is a new file, with no change to the loop.

### 7.2 Context objects

An enemy's update needs to steer around obstacles, query the ground height, damage the player and spawn bullets, all of which belong to the main module. Rather than give the enemy a reference back to the main module, a small plain object of exactly the callbacks and values it needs is rebuilt each frame and passed in. The same is done for power ups. The coupling stays one directional and the subclasses remain simple to reason about in isolation.

### 7.3 Frame rate independence

Every moving part (player, enemies, bullets, animations, timers, the camera) is advanced by the real time elapsed since the previous frame, never by a fixed per frame step. The game therefore

runs at the same speed regardless of the machine's frame rate.

## **7.4 Two lighting models in one scene**

The choice of material is deliberate per object and per dimension. The realistic dimension uses physically based materials, needed for the normal and roughness maps on the armour; the toon dimension uses cel shaded materials, which ignore those maps but give the flat, banded look. The two coexist in the same scene during the transition, each object carrying both and switching between them, and the emissive accents are shared by both because they read correctly either way.

## **7.5 Robustness**

Several measures keep the simulation stable: the per axis sliding and the depenetration fallback together guarantee that no character is ever left stuck inside an obstacle regardless of approach angle; the arena clamp guarantees nothing leaves the playable area even though the walls have no collider; the enemy list is iterated backwards so that removing a defeated enemy mid loop does not disturb the iteration; and the wave manager advances through the same code path whether a wave is cleared by the player or by the development skip key.